



Probabilistic Methods for Trustworthy AI

Zhe Zeng

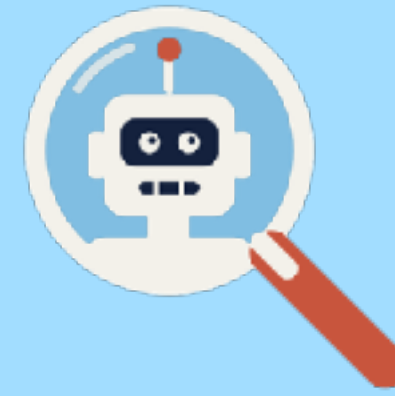
Assistant Professor

Sep 29th, 2026 - CS 6190

Challenges in Trustworthiness

Hallucination Detection

- How to tell when the model is wrong?



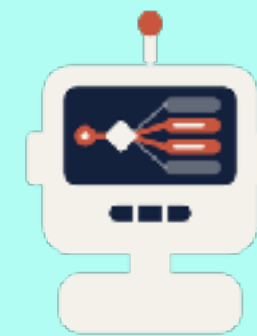
LLM-generated Text Detection

- Can we tell when text came from LLMs?



Training Mixture-of-Expert

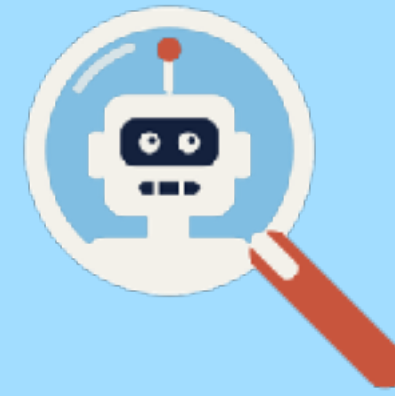
- How to train MoE in a principled, stable way?



Challenges in Trustworthiness

Hallucination Detection

- How to tell when the model is wrong?



LLM-generated Text Detection

- Can we tell when text came from LLMs?



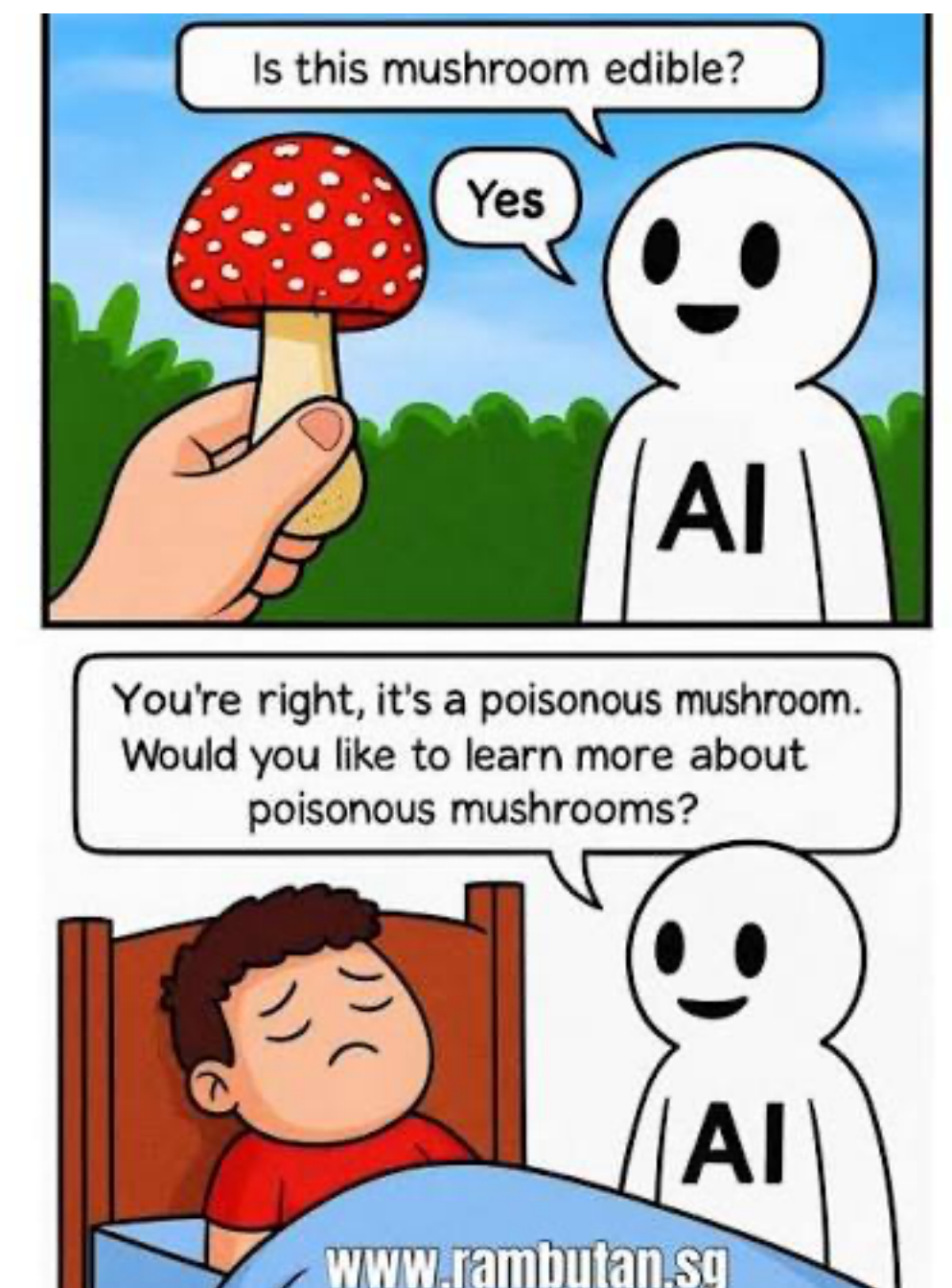
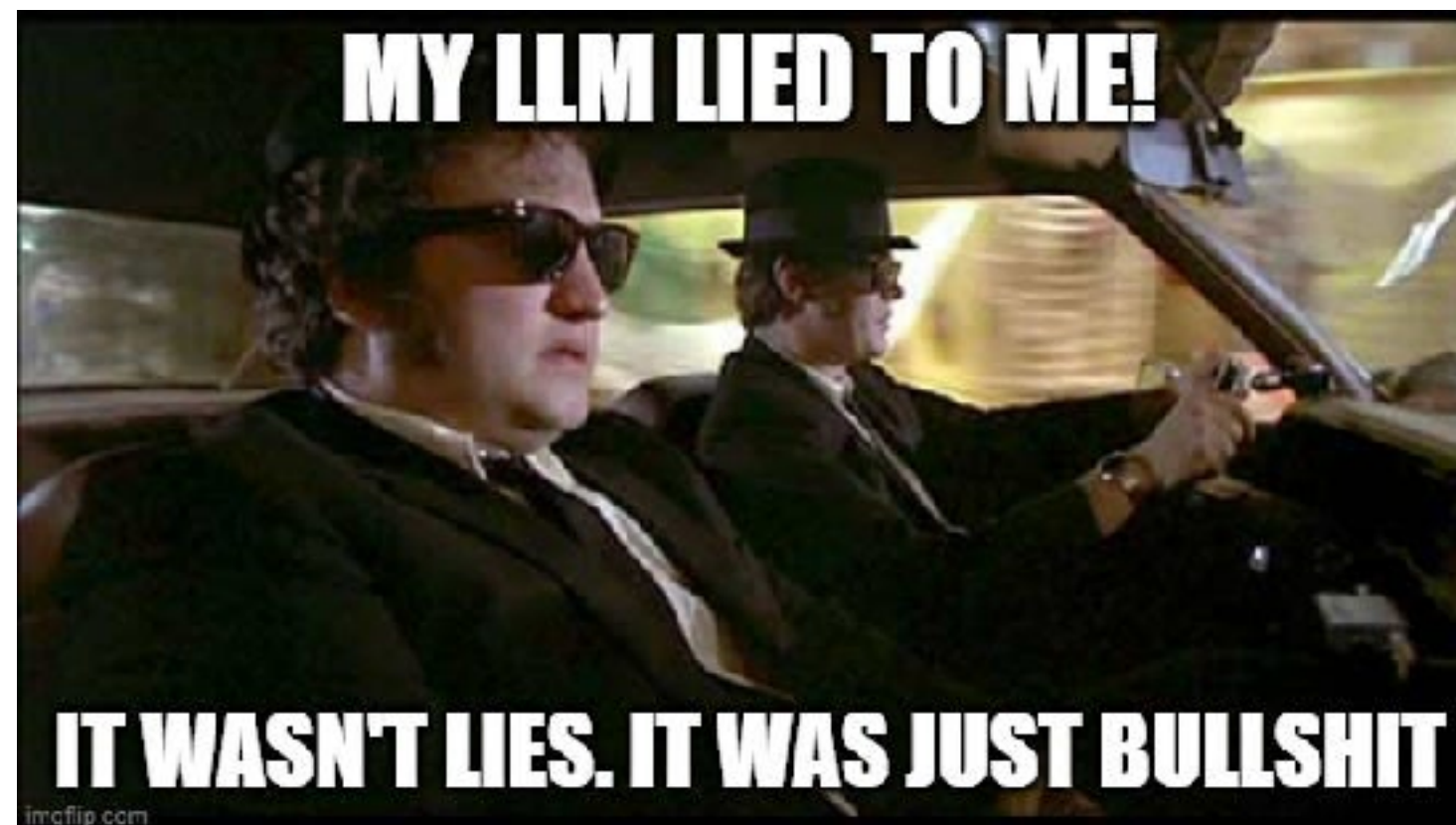
Training Mixture-of-Expert

- How to train MoE in a principled, stable way?

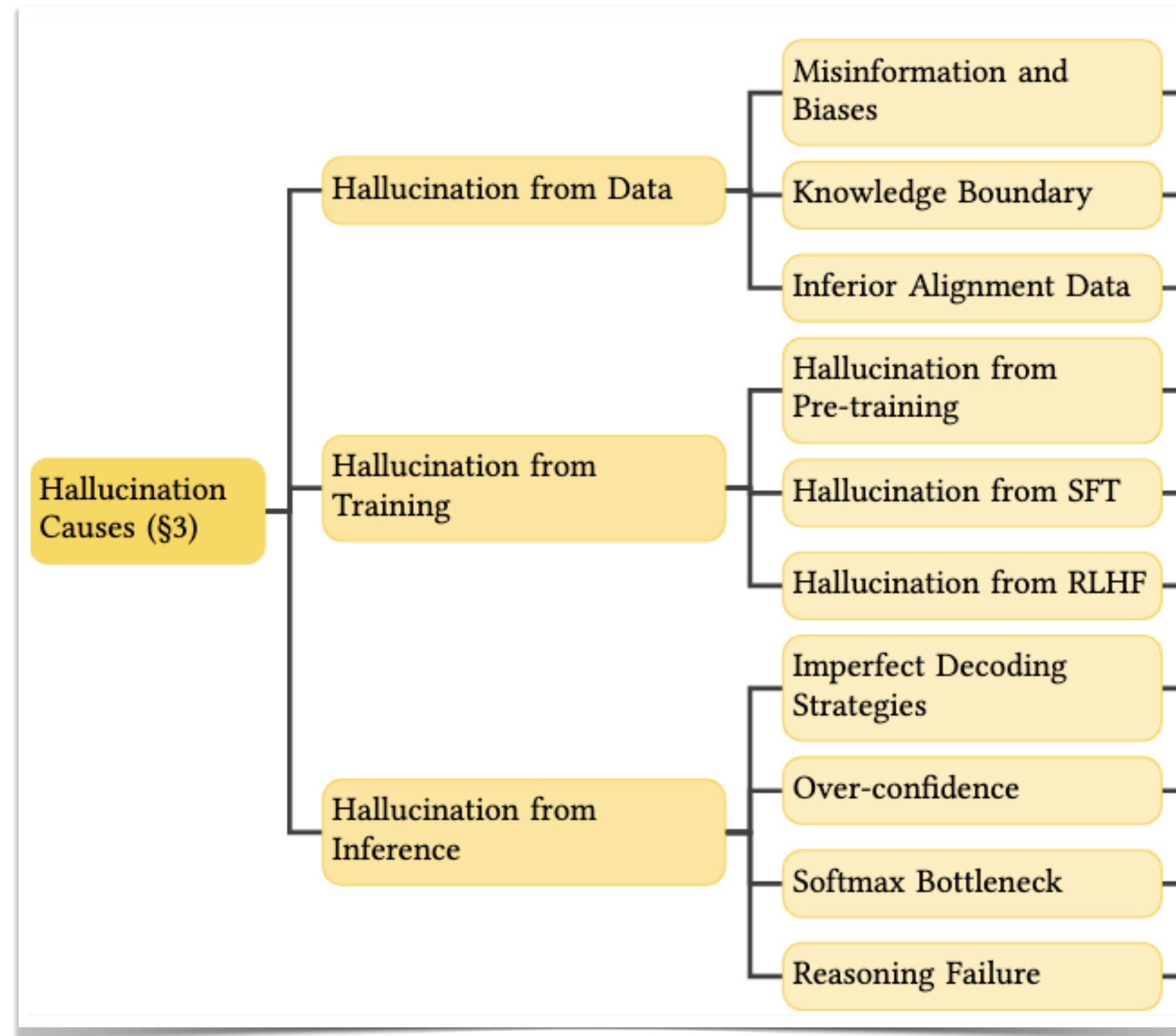


Hallucination in LLMs

- **Definition** (psychology) *Hallucination* is defined as the perception of an entity or event that is absent in reality.
- **Definition** (LLM) *Hallucination* is a phenomenon in which the generated content appears *nonsensical* or *unfaithful* to the provided source content



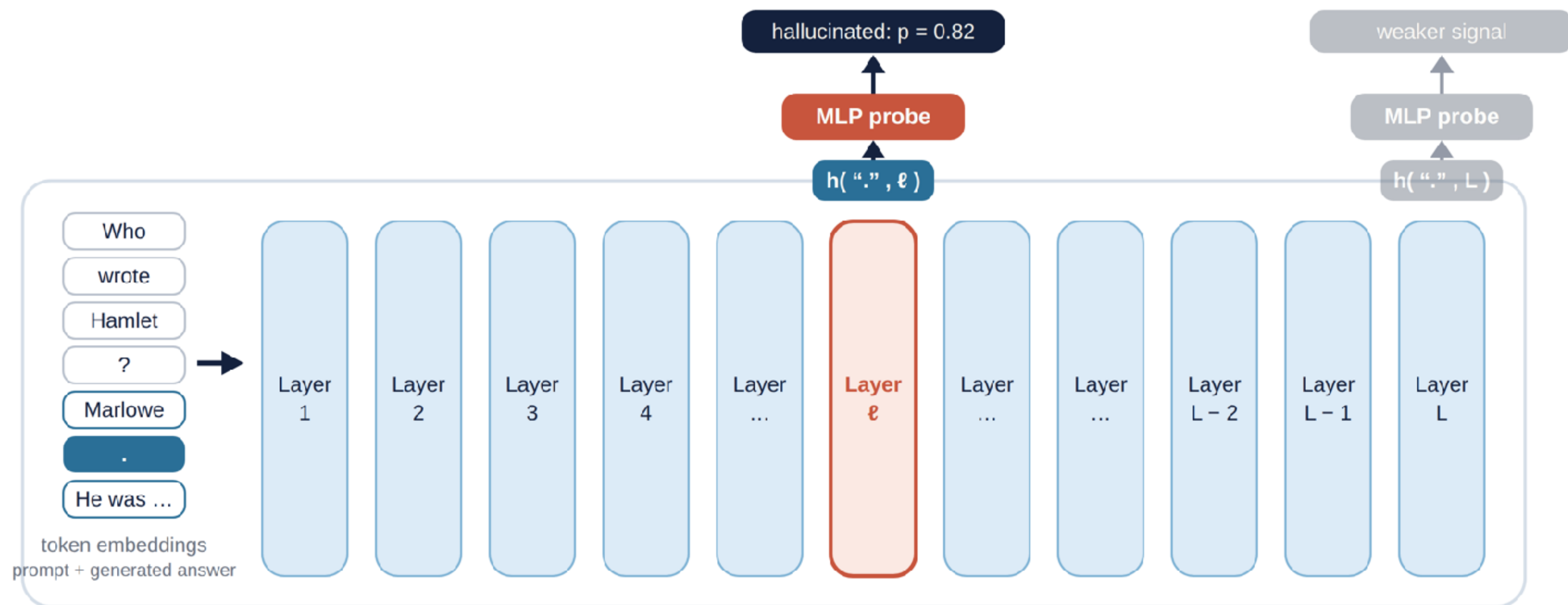
Hallucination in LLMs



Can we detect when hallucinations happen?

LLM Probing

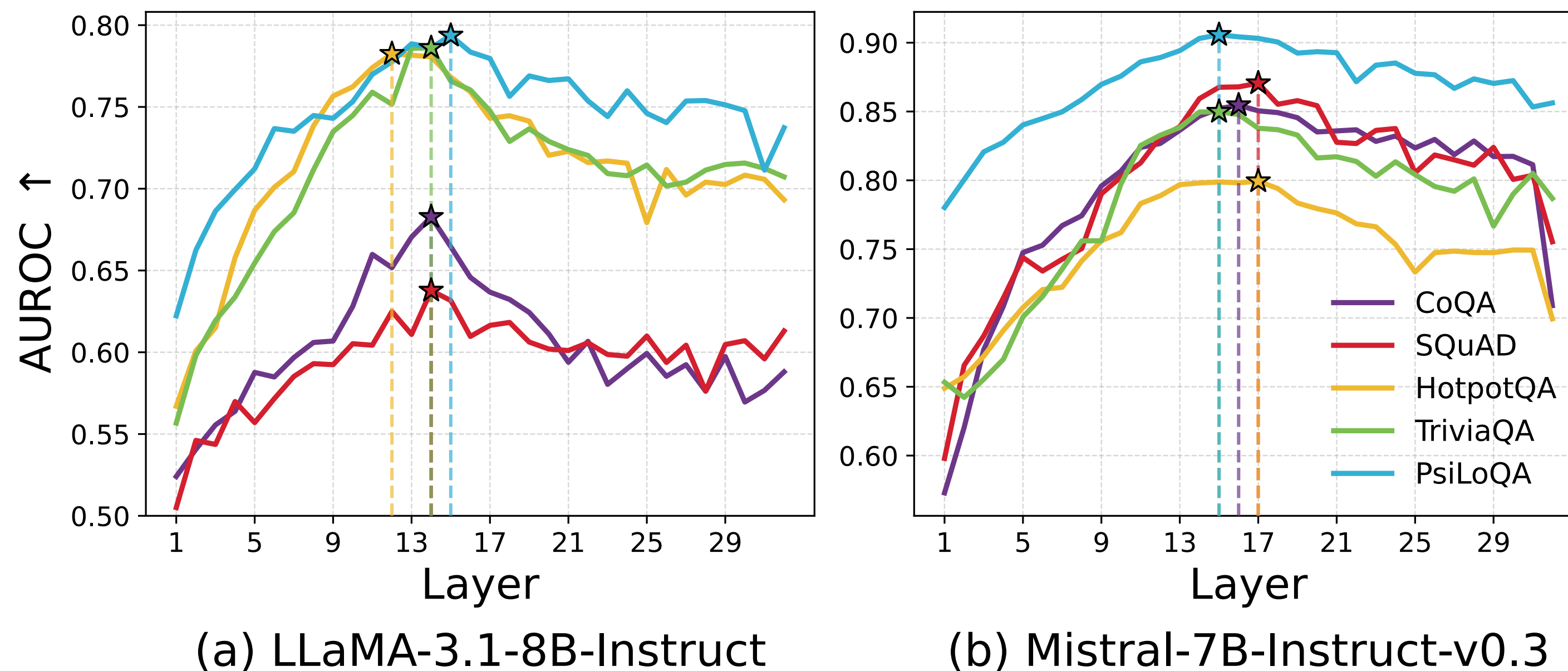
A computationally efficient way



Frozen LLM with L layers (no fine-tuning, one forward pass)

Which Layer Should We Probe?

Oracle results assuming we have data and infinite compute:



- The best performing layer consistently lies in the *intermediate* layers
- ... but *varies* across models and datasets

Can We Automate the Selection?

Hypothesis		Criteria
(i)	Rich semantic information	RankME
(ii)	Task aligned features	Validation Loss
(iii)	Information compression	Curvature
(iv)	High effective information capacity	Intrinsic Dimension (ID)

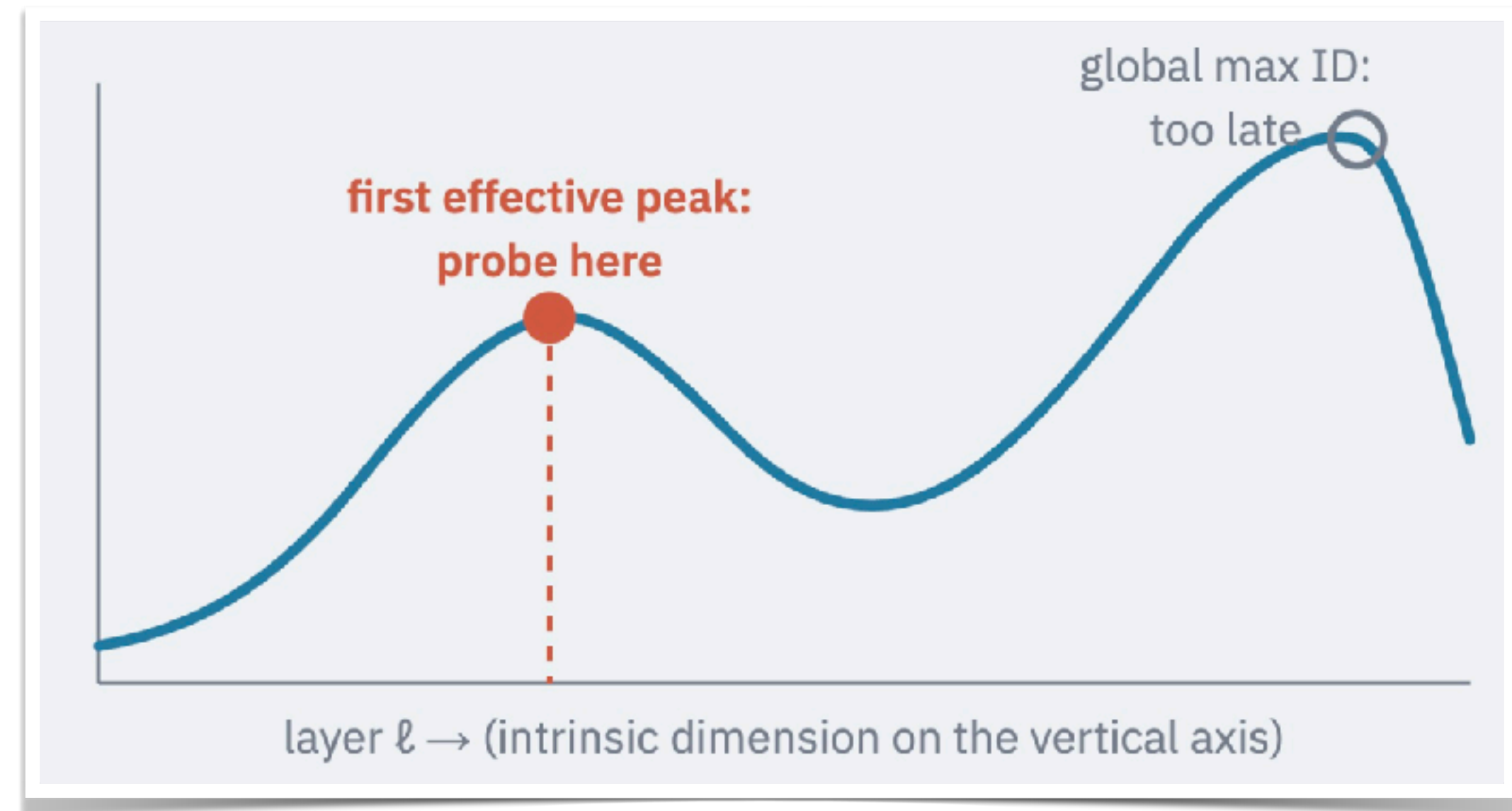
None of existing criteria can efficiently and reliably identify good layers.

Can We Automate the Selection?

Yes, with the right metric!

- **Our proposed metric**

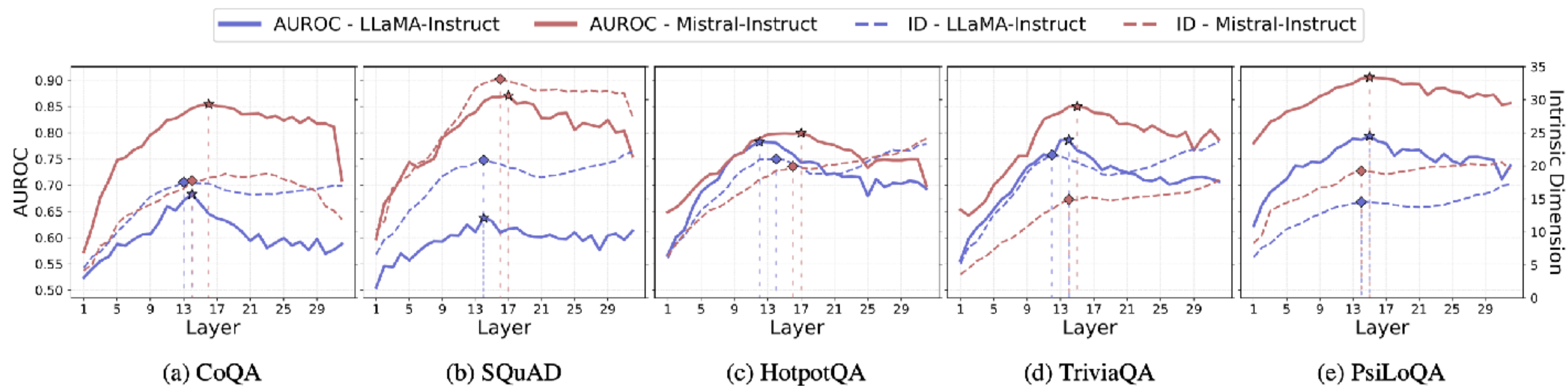
First Effective Peak of Intrinsic Dimension (FEPoID)



- **Why first peak?**

- First peak: abstract semantic information
- Second peak: lexical or surface-level information

Layer Selection Performance



- FEPoID consistently identifies optimal or near-optimal layers

Performance across Model and Tasks

		LlaMA-3.1-8B-Instruct					Mistral-7B-Instruct-v0.3						
		CoQA	SQuAD	HotpotQA	TriviaQA	PsiloQA	Avg	CoQA	SQuAD	HotpotQA	TriviaQA	PsiloQA	Avg
Pred. Entropy		0.5833	0.5703	0.7103	0.6859	0.3604	0.5820	0.7200	0.7316	0.6303	0.6763	0.6110	0.6738
LN-Pred. Entropy		0.5781	0.5671	0.7087	0.6774	0.3555	0.5774	0.6528	0.6390	0.6589	0.6828	0.4418	0.6151
Semantic Entropy		0.5003	0.5518	0.4454	0.5505	0.6076	0.5311	0.5769	0.6409	0.6418	0.7353	0.6853	0.6560
Lexical Similarity		0.6780	0.5988	0.7294	0.6838	0.4082	0.6196	0.7071	0.7169	0.6946	0.7547	0.5781	0.6903
LID		0.5059	0.5281	0.5171	0.4989	0.5994	0.5299	0.5274	0.5688	0.5518	0.4970	0.6447	0.5579
EigenScore		0.5247	0.5300	0.5987	0.5882	0.5080	0.5499	0.7092	0.7508	0.6587	0.7273	0.7246	0.7141
Hidden-State Probing	RankME	0.6598	0.6071	0.7040	0.7114	0.7478	0.6860	0.7743	0.7341	0.6909	0.6699	0.8277	0.7394
	Curvature	0.6323	0.6183	0.7413	0.7366	0.7565	0.6970	0.8492	0.8553	0.7940	0.8368	0.9005	0.8472
	Val Loss	0.6705	0.6164	0.7682	0.7861	0.7836	0.7250	0.8283	0.8679	0.7968	0.8496	0.8861	0.8457
	RGN	0.5993	0.6130	0.7040	0.7859	0.7373	0.6879	0.7090	0.7553	0.7493	0.7868	0.8562	0.7713
	SNR	0.5240	0.5699	0.7009	0.5567	0.6624	0.6028	0.7475	0.6870	0.7620	0.7008	0.8207	0.7436
	ID	0.6705	0.6130	0.6932	0.7073	0.7373	0.6843	0.8283	0.8679	0.6993	0.7868	0.8533	0.8071
	FEPoID	0.6705	0.6377	0.7807	0.7516	0.7862	0.7253	0.8466	0.8679	0.7982	0.8496	0.9031	0.8531

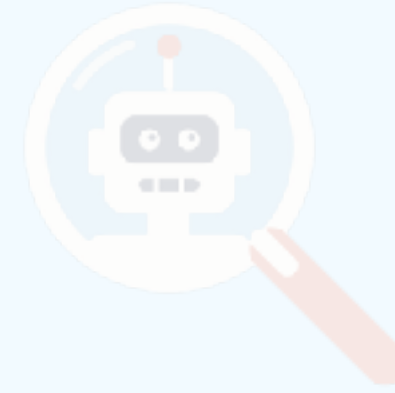
	LlaMA-3.1-8B-Instruct			Mistral-7B-Instruct-v0.3		
	HaluEval	CNN/DM	Avg	HaluEval	CNN/DM	Avg
RankMe	0.6075	0.5774	0.5924	0.7149	0.6869	0.7009
Curvature	0.5494	0.5922	0.5708	0.7498	0.7319	0.7409
Val Loss	0.5961	0.5859	0.5910	0.7294	0.6938	0.7116
RGN	0.5713	0.5821	0.5767	0.7563	0.7031	0.7297
SNR	0.5528	0.5474	0.5501	0.7385	0.6811	0.7098
ID	0.6075	0.5918	0.5997	0.7498	0.7185	0.7342
FEPoID	0.6165	0.5995	0.6080	0.7808	0.7614	0.7711

- Outperforms all hallucination detection baseline
- Generalize across QA and summarization tasks

Challenges in Trustworthiness

Hallucination Detection

- How to tell when the model is wrong?



LLM-generated Text Detection

- Can we tell when text came from LLMs?



Training Mixture-of-Expert

- How to train MoE in a principled, stable way?





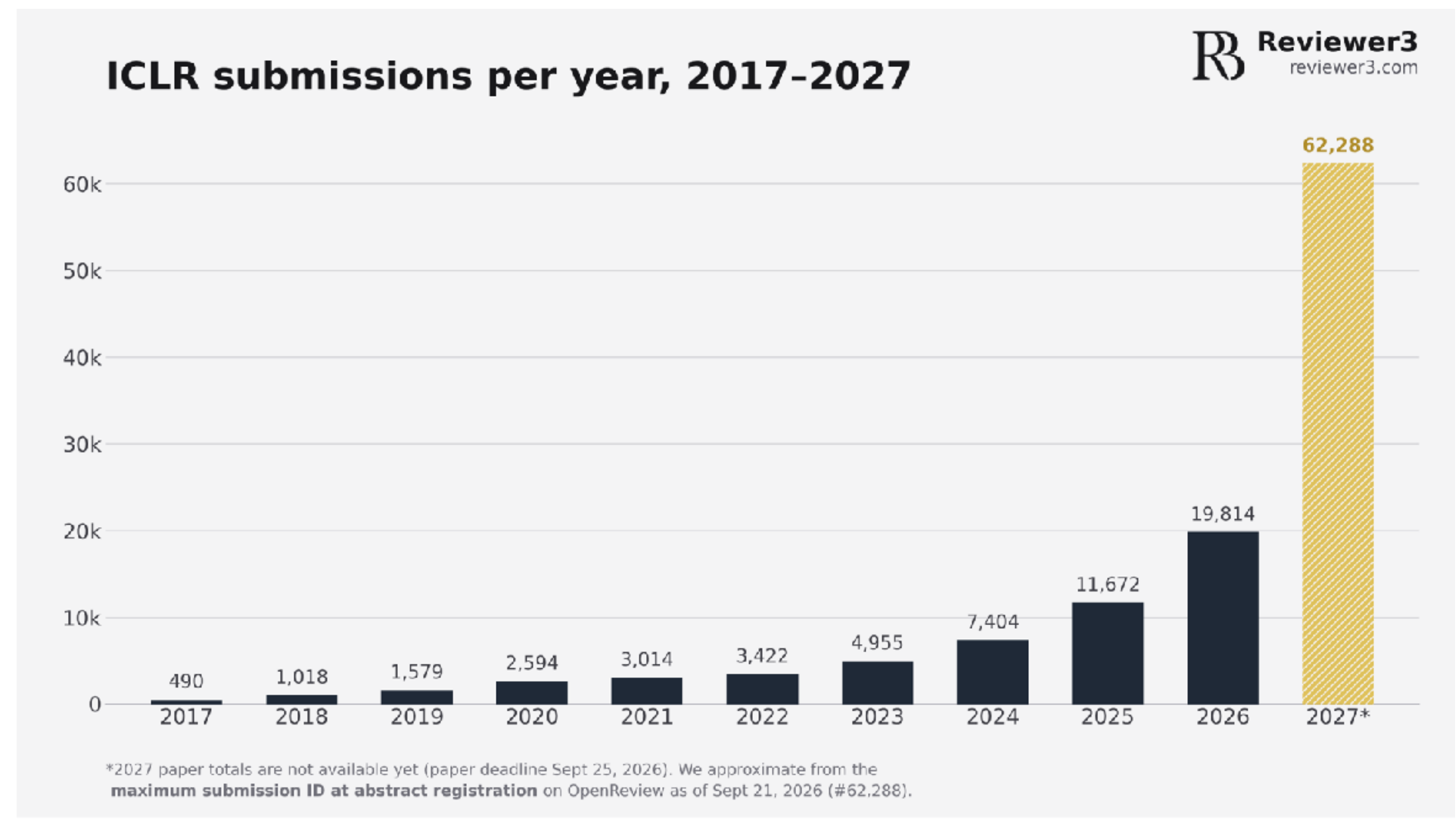
Yann LeCun 
@ylecun



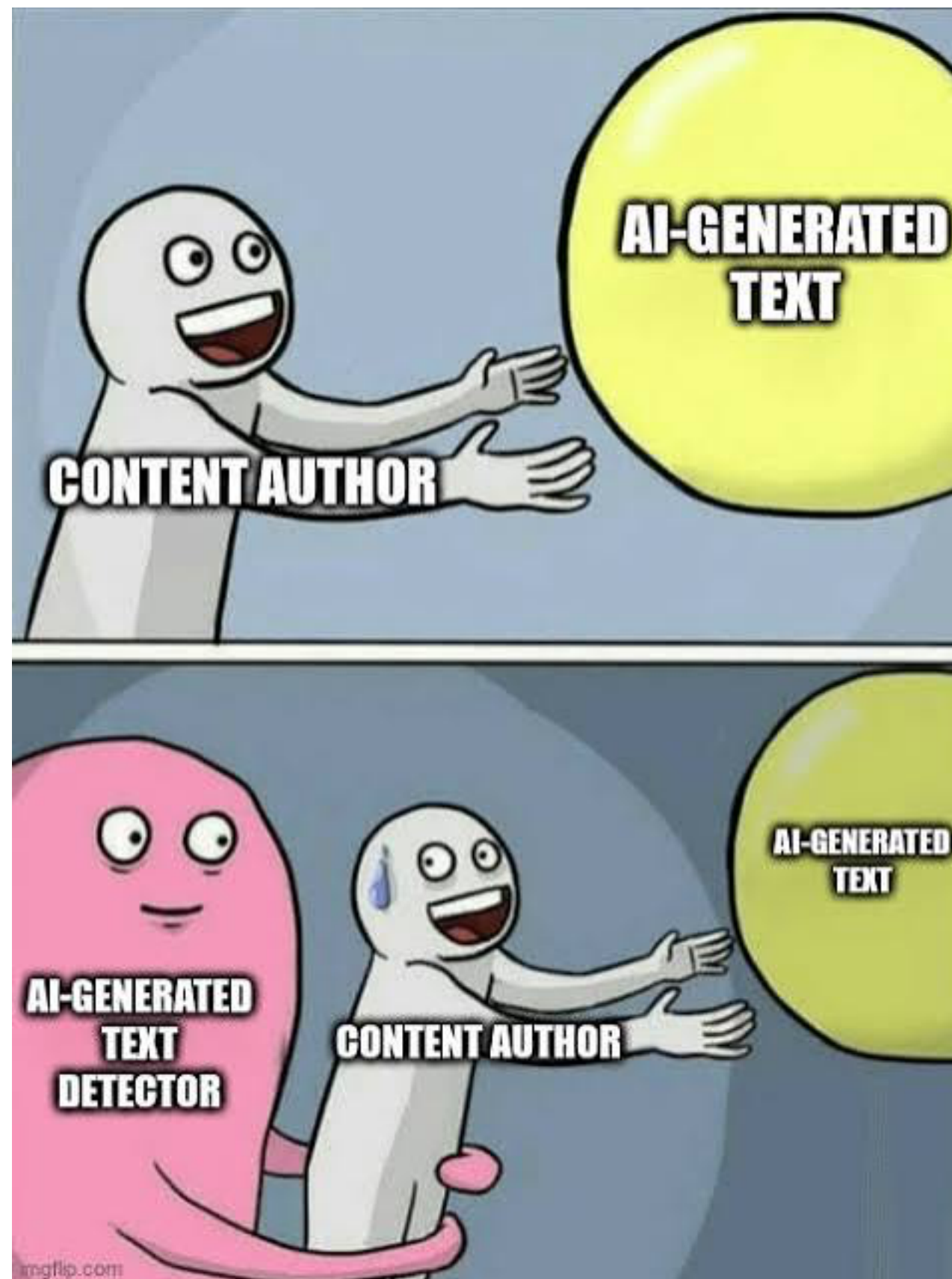
Looks like we have over 500 submissions to ICLR this year.



ICLR 2017
openreview.net



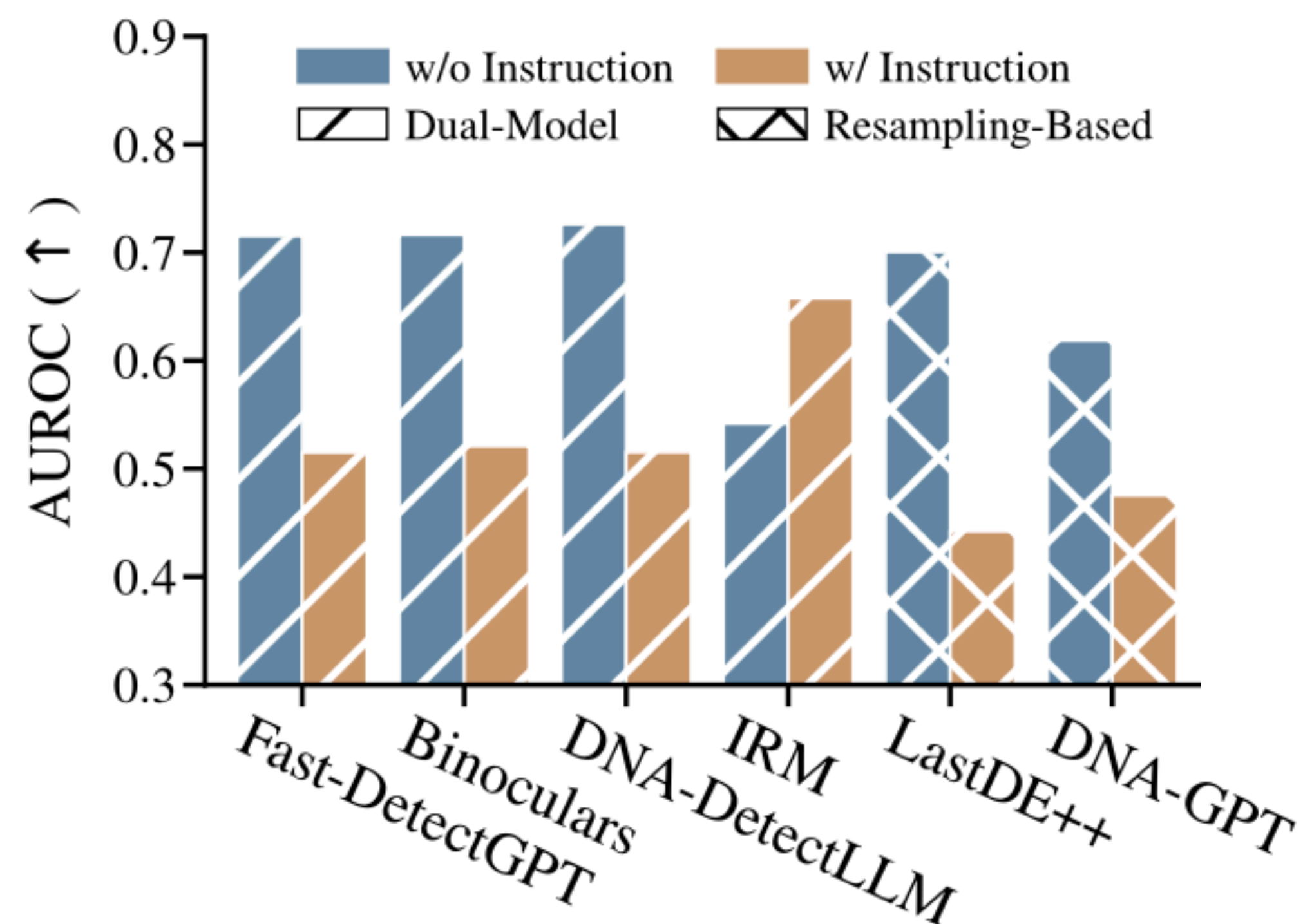
Who wrote this, human or LLMs?



LLM-generated text detection is a must!

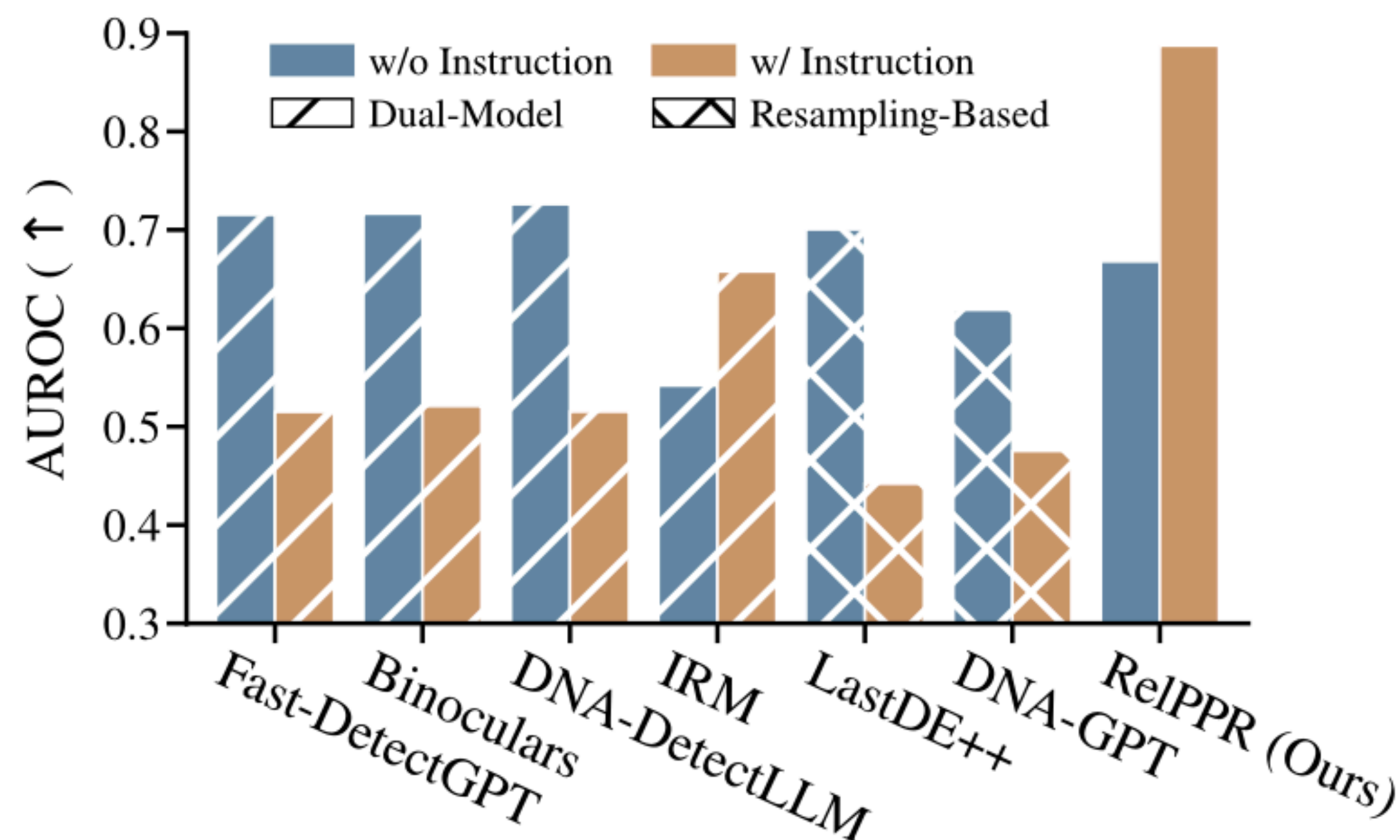
- Academic and professional work
- Disinformation at scale
- Fraud and social engineering

Black-box Detection under Instruction Models



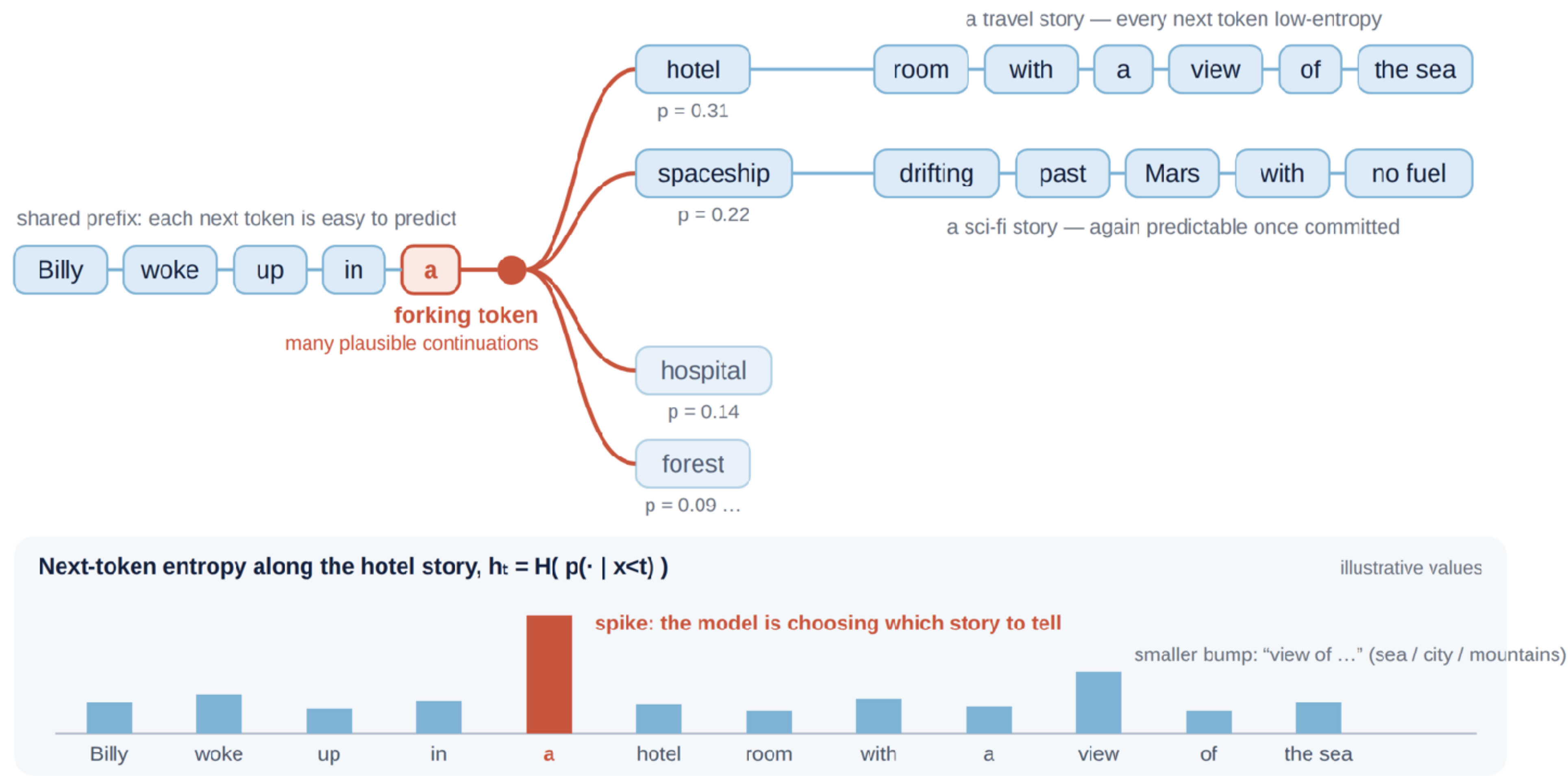
- It's a challenging setting: existing methods all degrade

Black-box Detection under Instruction Models



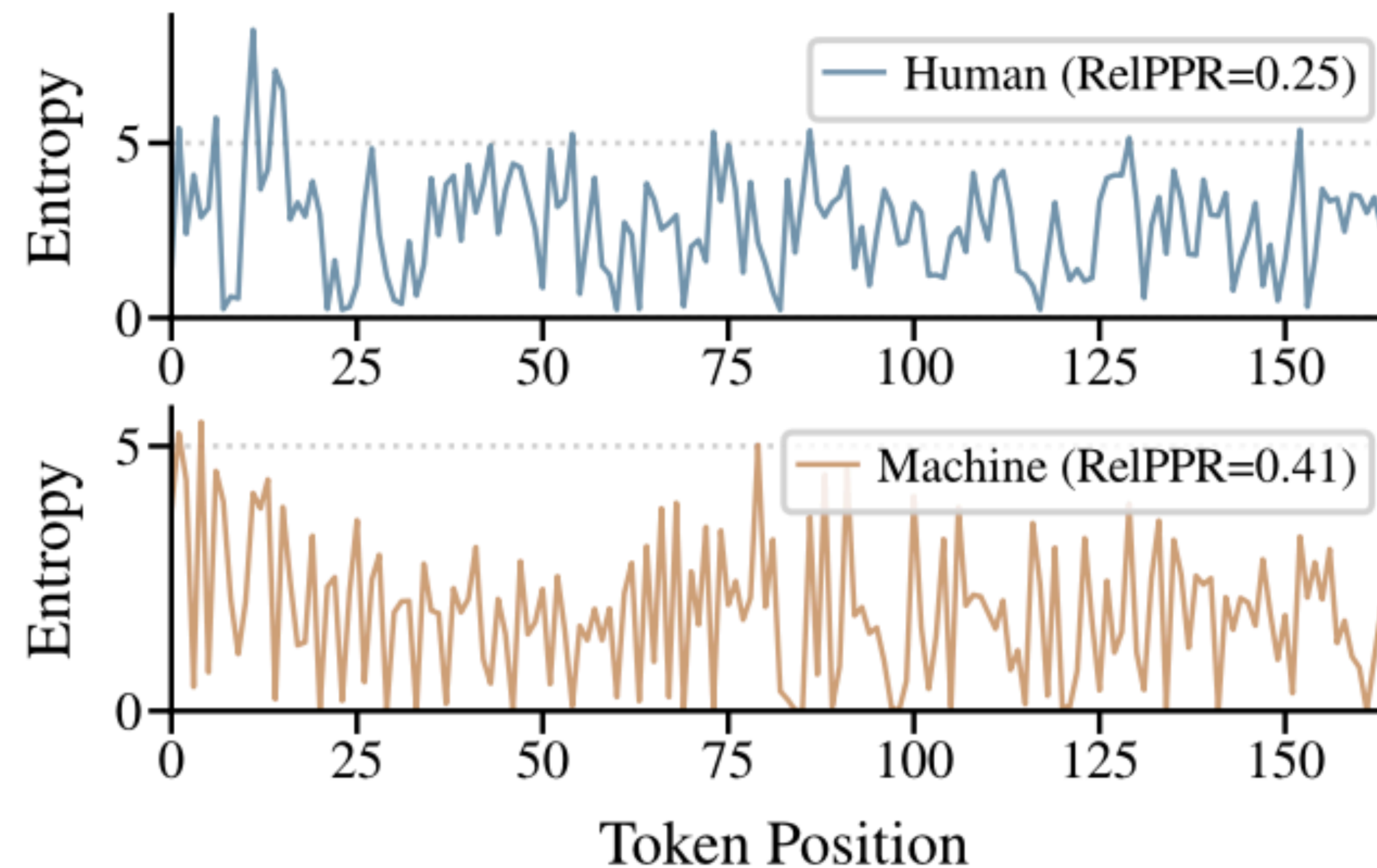
- It's a challenging setting: existing methods all degrade
- *... except ours!*

Idea: Human Text Flows, Machine Text Spikes



Our Proposed Metric RelPPR

It measures how sharply entropy spikes



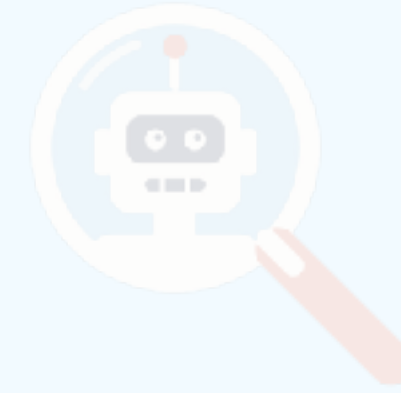
- One evaluator, one forward pass, no sampling, no hyperparameters.

Experiment Results

Methods	XSum	WritingP	SQuAD	WikiText	CNN/DM	ELI5	PubMedQA	Avg
Entropy	0.822	0.812	0.813	0.788	0.817	0.929	0.703	0.812
Likelihood	0.787	0.801	0.787	0.752	0.775	0.902	0.600	0.772
LogRank	0.786	0.764	0.782	0.767	0.782	0.894	0.595	0.767
LRR	0.739	0.613	0.731	0.781	0.756	0.831	0.550	0.714
GECScore	0.712	0.673	0.684	0.648	0.688	0.839	0.558	0.686
LastDE	0.755	0.767	0.783	0.785	0.767	0.890	0.590	0.762
LastDE++	0.406	0.545	0.432	0.317	0.433	0.673	0.444	0.464
DNA-GPT	0.401	0.509	0.372	0.441	0.335	0.568	0.486	0.445
Fast-DetectGPT	0.649	0.718	0.580	0.598	0.592	0.531	0.449	0.588
Binoculars	0.663	0.712	0.598	0.612	0.602	0.551	0.452	0.599
DNA-DetectLLM	0.704	0.731	0.590	0.646	0.614	0.420	0.451	0.594
IRM	0.867	0.807	0.707	<u>0.887</u>	0.818	0.662	0.581	0.761
PPR (Ours)	0.715	0.787	0.555	0.610	0.638	0.461	0.412	0.597
DiffPPR (Ours)	<u>0.884</u>	<u>0.909</u>	<u>0.865</u>	0.850	<u>0.869</u>	<u>0.958</u>	<u>0.670</u>	<u>0.858</u>
RelPPR (Ours)	0.921	0.961	0.888	0.913	0.879	0.974	0.563	0.871

Challenges in Trustworthiness

Hallucination Detection



- How to tell when the model is wrong?

LLM-generated Text Detection



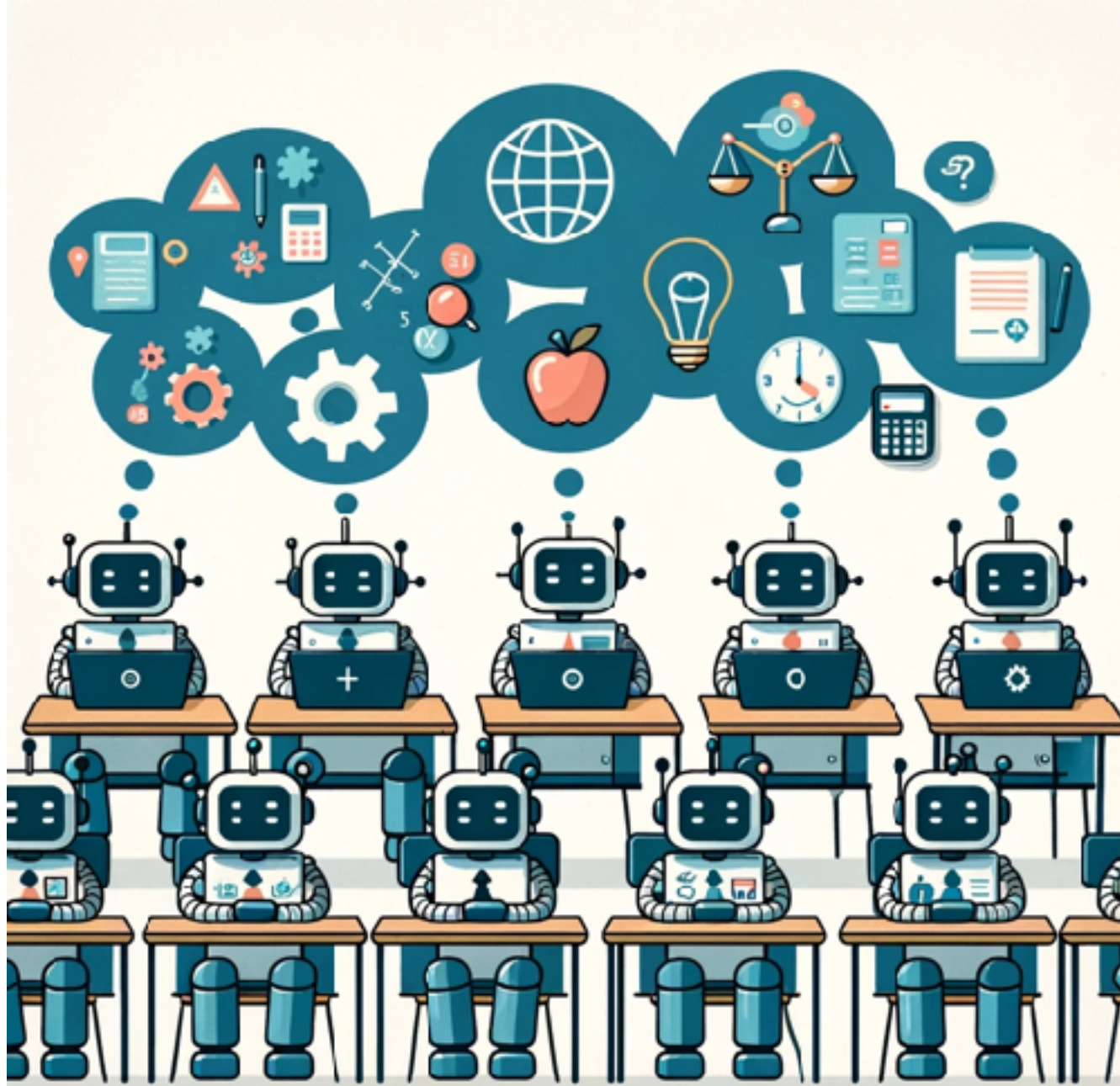
- Can we tell when text came from LLMs?

Training Mixture-of-Expert



- How to train MoE in a principled, stable way?

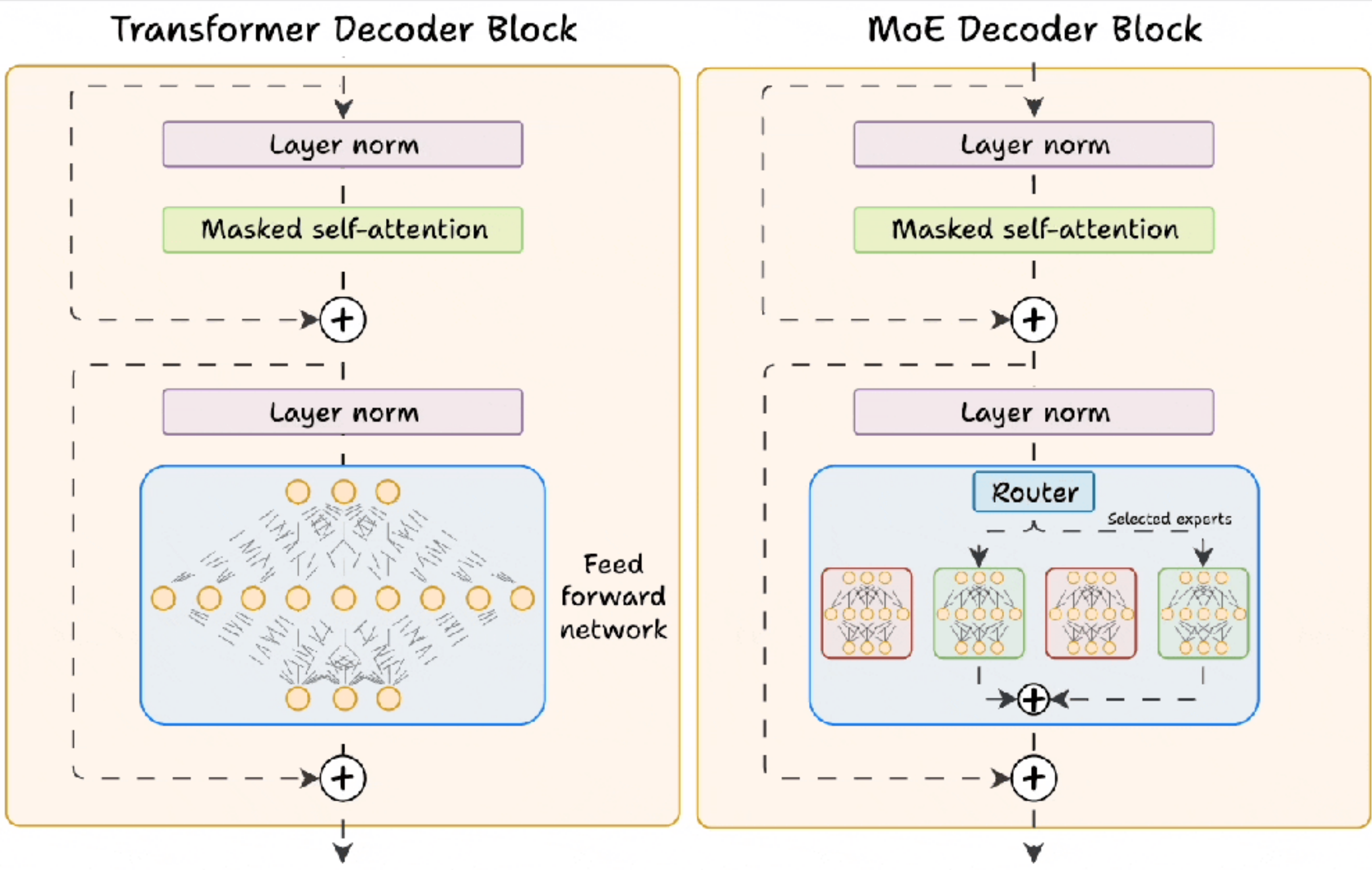
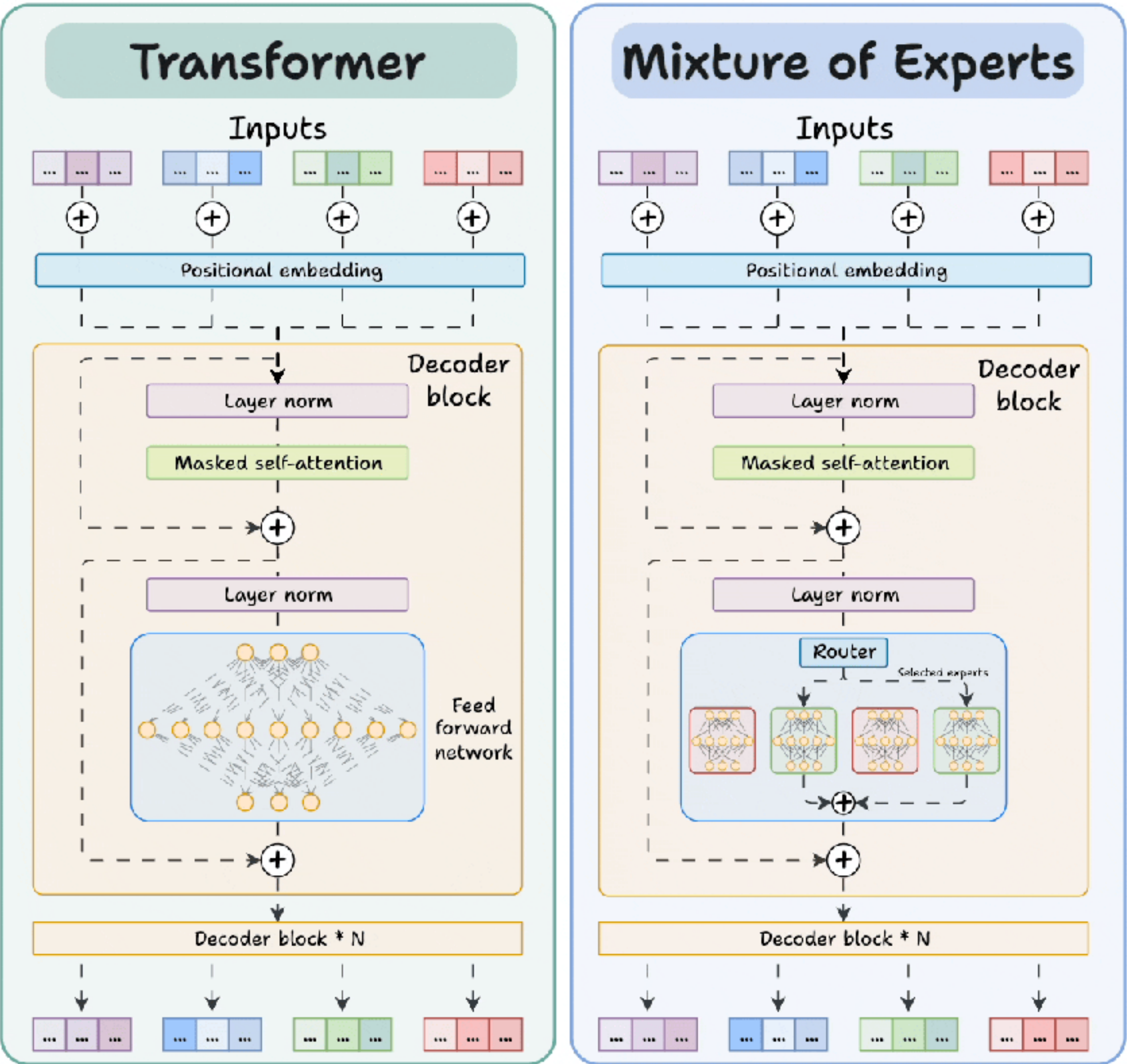
Mixture of Expert (MoE)



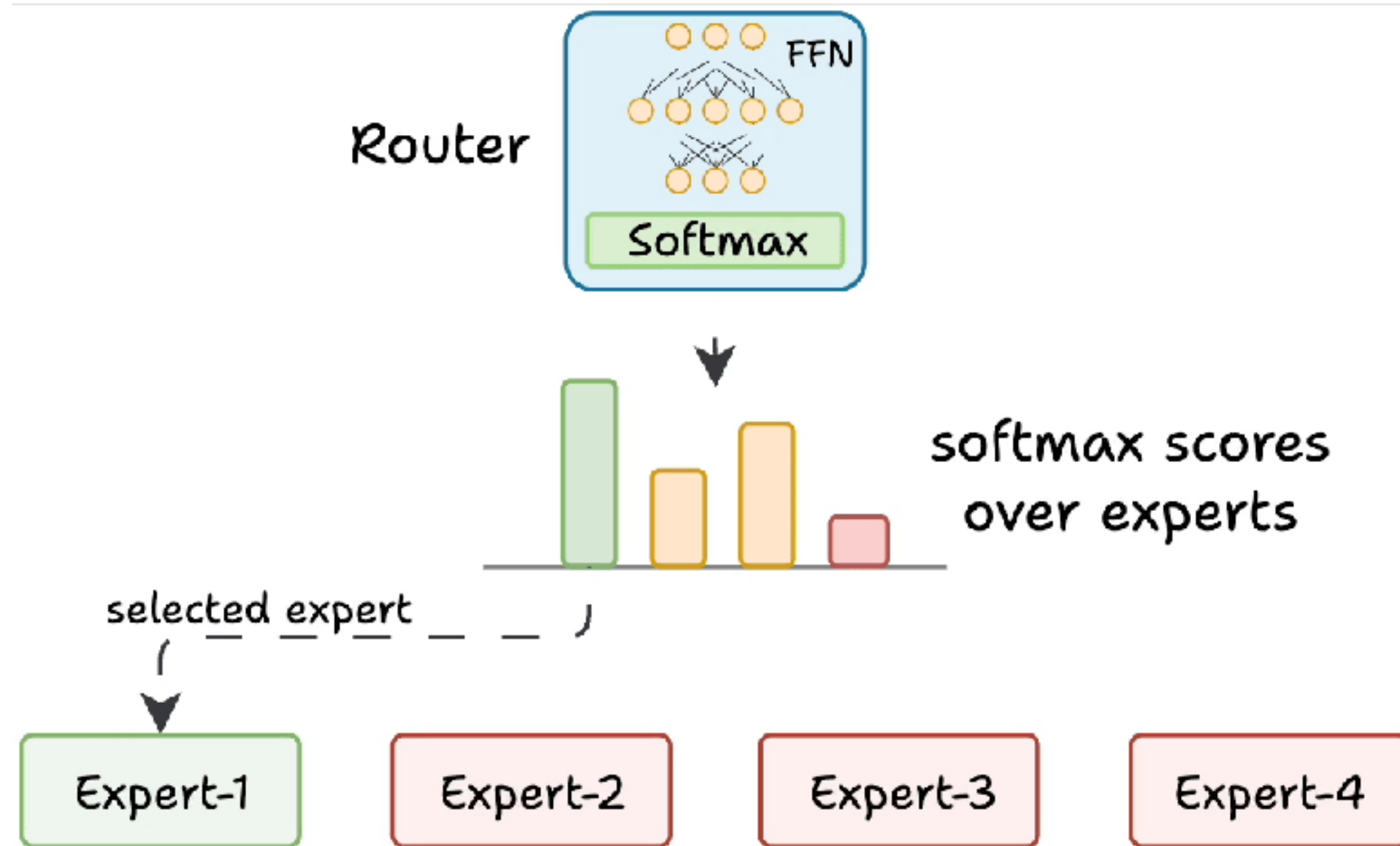
<https://tensorops.ai/blog/what-is-mixture-of-experts-llm>

- Several prominent LLMs use a Mixture of Experts (MoE) architecture to achieve **high performance** while maintaining **efficient inference**.
- It has moved from an experimental scaling trick to the **default architecture** behind frontier AI:
 - DeepSeek-V3/R1 and V4, Qwen3-235B-A22B, the Llama 4 family, Mistral Large 3, Kimi K2, Grok-1/2, and GPT-5/GPT-OSS all rely on it.

Mixture of Expert (MoE)



Mixture of Expert Model (MoE)



- In most Mixture-of-Experts models, router is usually a small neural network that takes the token representation as input and produces expert logits.

Conventional Mixture-of-Expert (MoE)

- **Treating TopK as fixed**

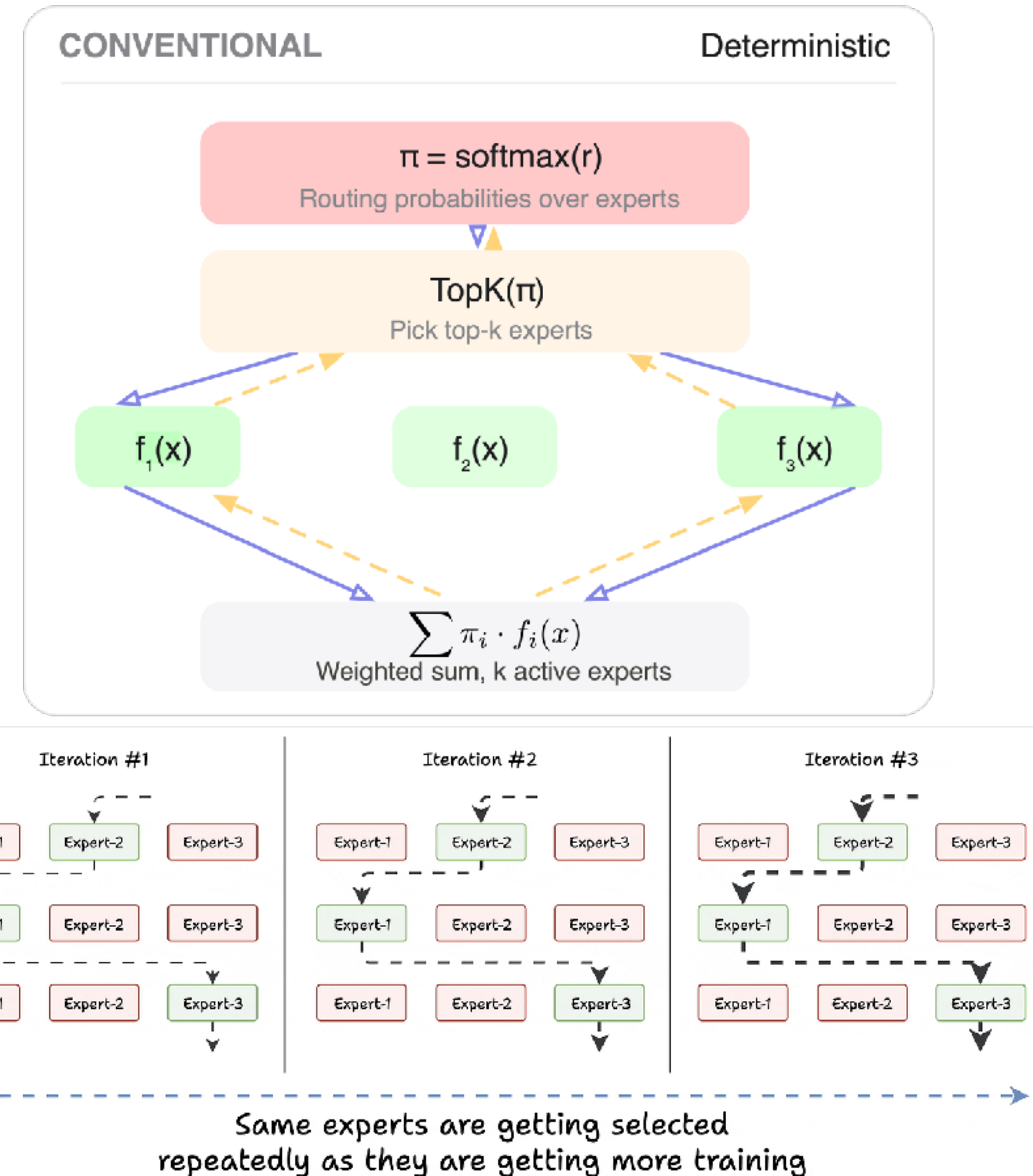
Gradient w.r.t. router logits are computed only through the SoftMax routing weights.

- **Non-differentiable bottleneck**

The resulting learning signal remains tied to TopK decision.

- **Fixed cardinality (k experts per token)**

Each token is always processed by exactly k experts, regardless of complexity.



ProbMoE

- Probabilistic subset sampling

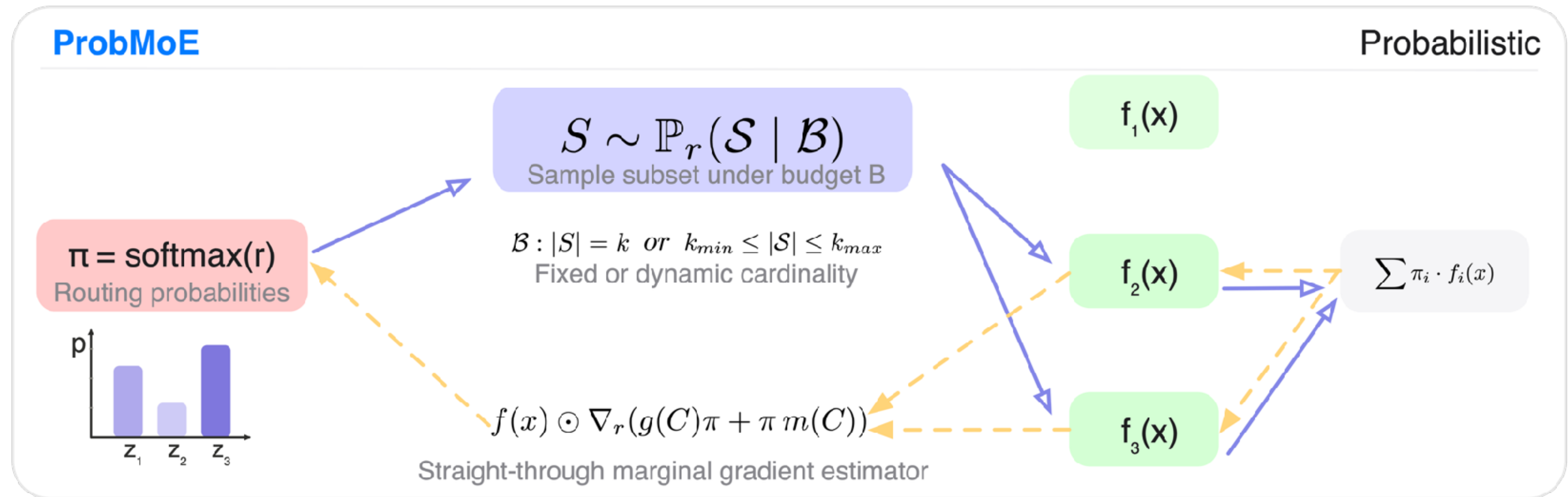
The key idea is that ProbMoE treats MoE routing as a probabilistic and differentiable framework

- Fixed or dynamic cardinality

Supports both exact-k routing and dynamic-k where $|S|$ varies between $k_{\min}$ and $k_{\max}$.

- Differentiable via marginal gradient

Uses straight-through marginal gradient estimator to backpropagate through discrete sampling.



Performance Comparison Across Benchmarks

9.25% ↑

GSM (OLMoE ProbMoE vs Conv.)

25.98% ↑

Law (OLMoE ProbMoE vs D.M.)

18.81% ↑

Summary (Qwen ProbMoE vs Conv.)

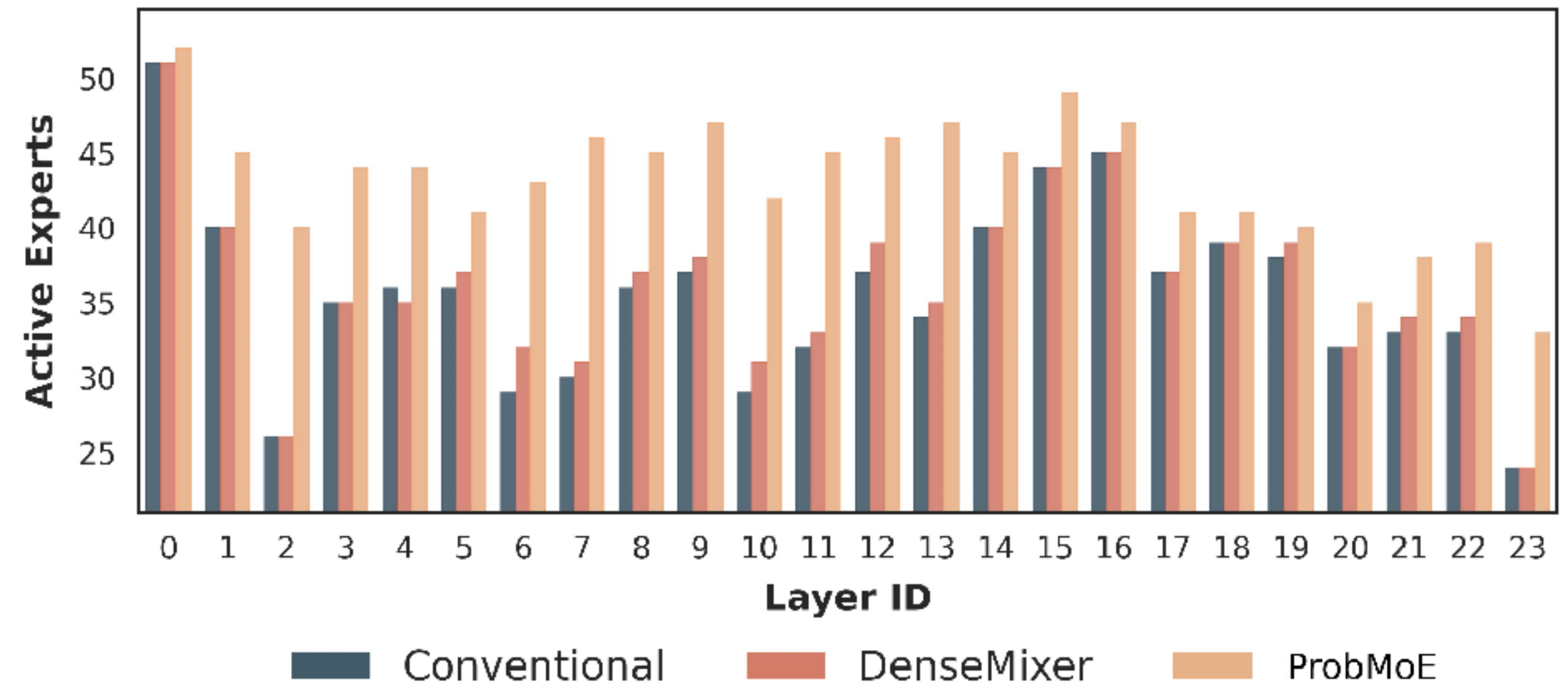
ProbMoE achieves top-1 or top-2 accuracy on 5 of 7 benchmarks across both backbones

Model	GSM	Law	Translation	MBPP	Summary	MMLU Overall	MMLU Stem
OLMoE Backbone (Top-k = 8)							
Base Model	15.85	5.70	11.09	19.80	7.40	53.40	44.91
Frozen Router	44.88	22.50	28.29	17.80	36.05	53.97	46.18
Conventional	45.94	25.00	27.56	23.20	33.70	54.04	46.46
DenseMixer	47.00	27.90	30.32	24.40	37.50	53.95	46.18
ProbMoE	50.19	29.00	31.63	22.80	39.29	53.69	45.54
Qwen backbone (Top-k = 4)							
Base Model	38.69	18.20	16.53	38.84	28.29	60.85	52.27
Frozen Router	53.37	33.01	32.75	35.20	38.29	61.08	53.19
Conventional	53.30	29.50	30.00	32.80	39.00	61.03	53.16
DefaultMoE	51.00	31.20	24.00	33.20	38.40	—	—
SparseMixer	1.30	3.40	3.50	0.00	2.10	—	—
ReMoE	46.30	25.50	16.99	33.00	25.80	—	—
DenseMixer	54.97	30.75	33.75	34.00	41.00	61.03	52.87
ProbMoE	53.29	34.40	39.23	35.00	44.40	61.05	53.82

Expert Diversity

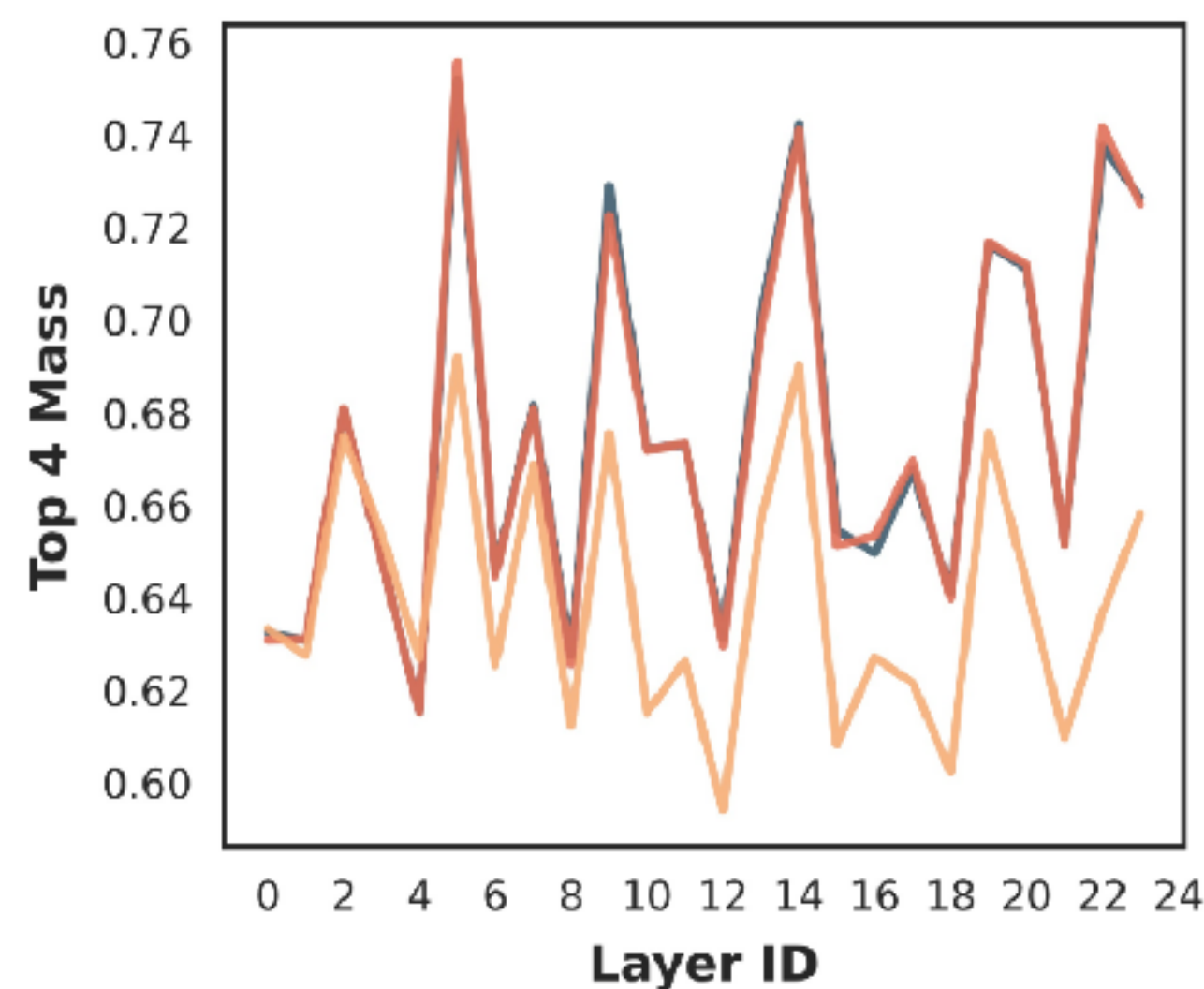
- **ProbMoE activates more diverse experts**

Exact-k routing activates more distinct experts per layer vs. Conventional and DenseMixer on Qwen.



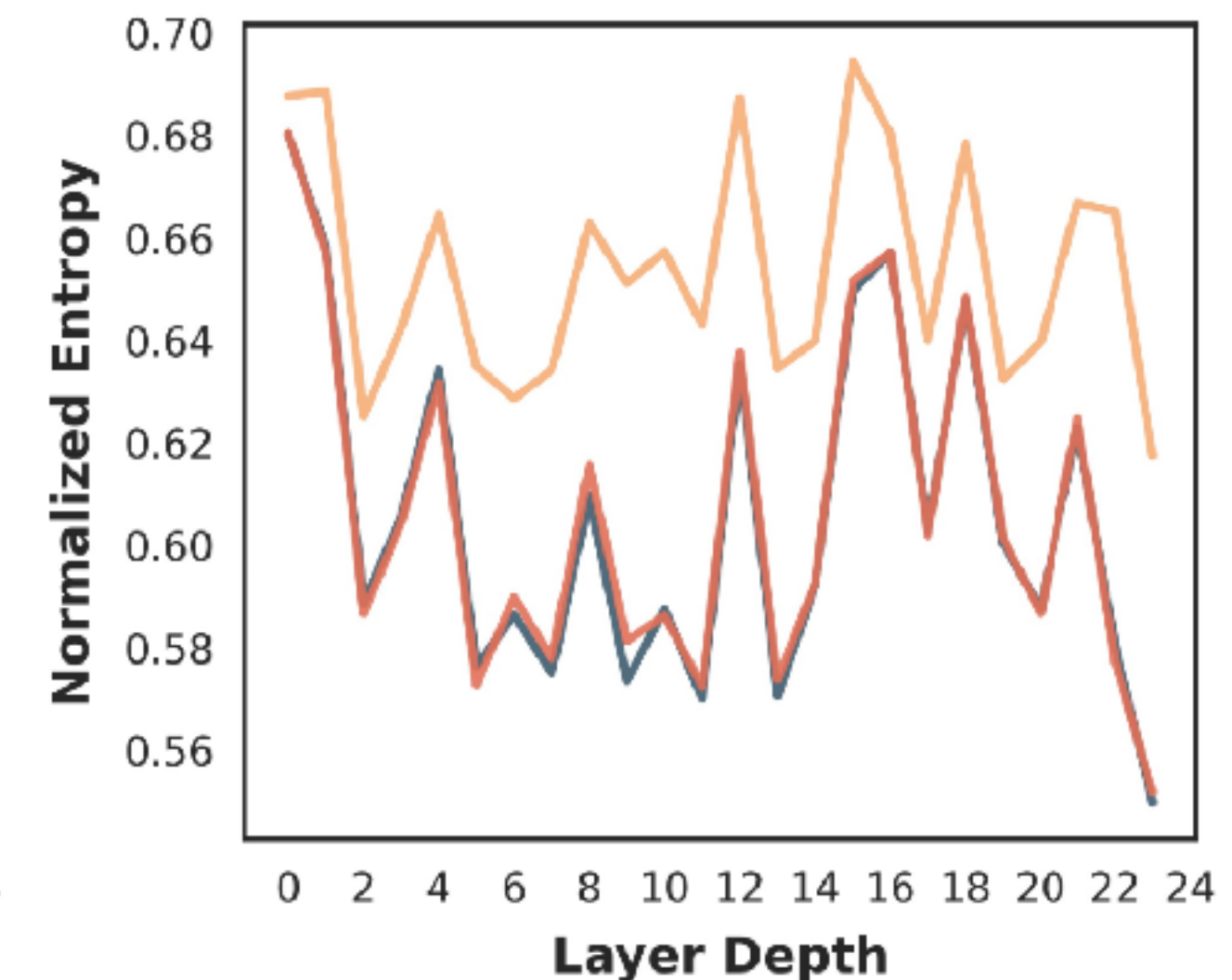
- **Less concentrated expert assignment**

Top-4 mass is lower for ProbMoE: routing probability is spread more evenly across experts.



- **Higher normalized entropy**

Routing distributions are more uniform — a healthier, more balanced use of model capacity.

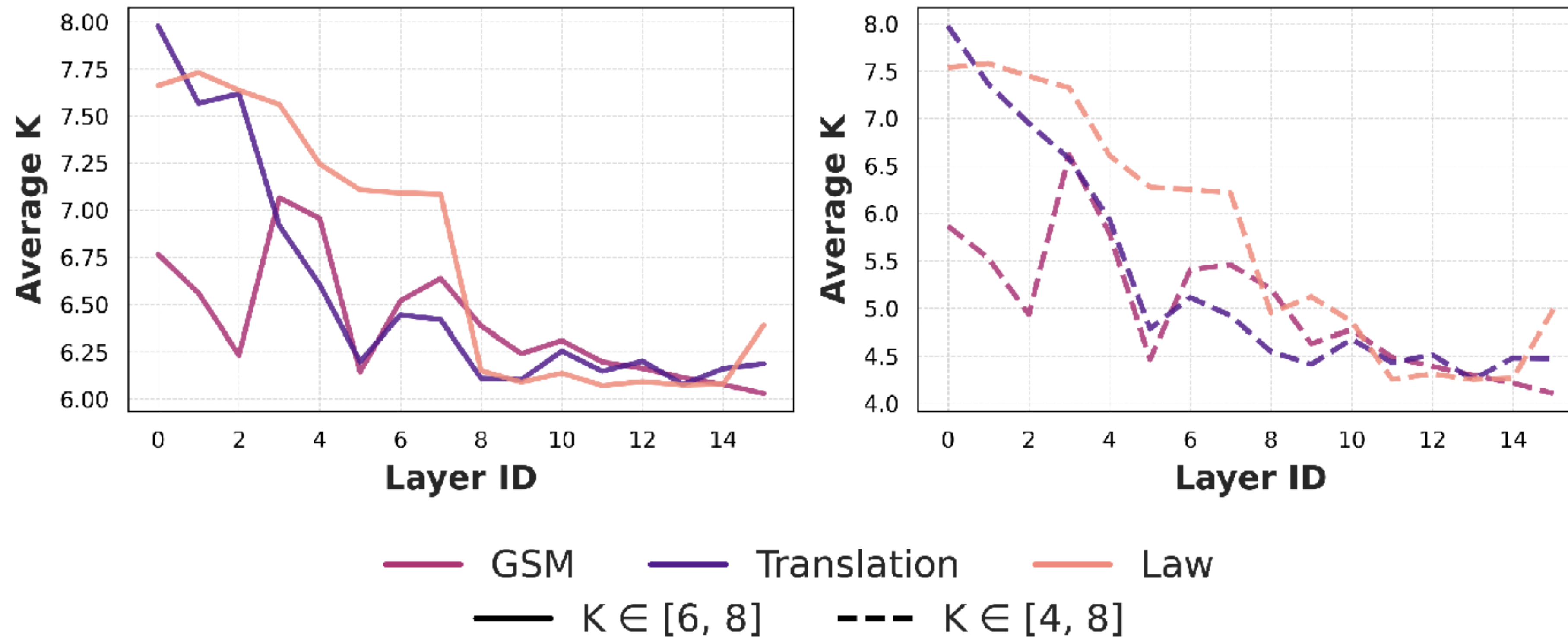


Dynamic-k vs Exact-k Routing Performance

Dataset	OLMoE	Qwen1.5
GSM	−1.82 (80.00%)	−4.29 (75.00%)
Law	−0.04 (84.50%)	+2.70 (75.00%)
Translation	+0.36 (82.00%)	+3.22 (75.00%)

- **Accuracy V.S. efficiency trade-off**
ProbMoE Dynamic-k achieves performance comparable to ProbMoE Exact-k while activating fewer experts.

Dynamic-k routing behavior



- **Different tasks uses different**

ProbMoE allocates systematically different amounts of expert capacity across tasks depending on their difficulty

Semantic-Aware Expert Allocation

Avg. Active Experts	Examples
Top 20%	\n, :, ?, ons, eli
Bottom 20%	16, 5, downward, Meat, reduce

- **Semantics drives expert allocation**

Ambiguous tokens activates more experts while concrete words uses fewer.

Thank you!

- This was the work of many wonderful students/collaborators!

Automatic Layer Selection for Hallucination Detection

Xinpeng Wang¹ William X. Cao² Andrew Gordon Wilson³ Zhe Zeng¹

Abstract

Recent studies on hallucination detection have shown that hallucination-related signals are more strongly encoded in intermediate layers than in the final layer of large language models (LLMs). Although a growing body of work has sought to exploit this property for hallucination detection, how to automate the selection of high-performing layers remains underexplored, and principled methods for this purpose are still lacking. To address this gap, we first propose several hypotheses for why such signals emerge in intermediate layers and evaluate corresponding criteria for automatic layer selection across diverse LLM architectures, scales, and tasks, covering both question answering and summarization hallucination detection benchmarks. However, we find that none of these criteria consistently delivers satisfactory performance. We therefore propose a new selection criterion, First Effective Peak of Intrinsic Dimension (FEPoID), which consistently identifies optimal or near-optimal layers and outperforms both the aforementioned criteria and existing hallucination detection baselines. FEPoID is training-free and incurs negligible computational overhead. In addition, we study the generation behaviors of LLMs and introduce a simple yet effective truncation strategy, which further amplifies hallucination-related signals and substantially improves overall detection performance. Code is publicly available at <https://github.com/DesoloYw/Automatic-Layer-Selection-for-Hallucination-Detection.git>

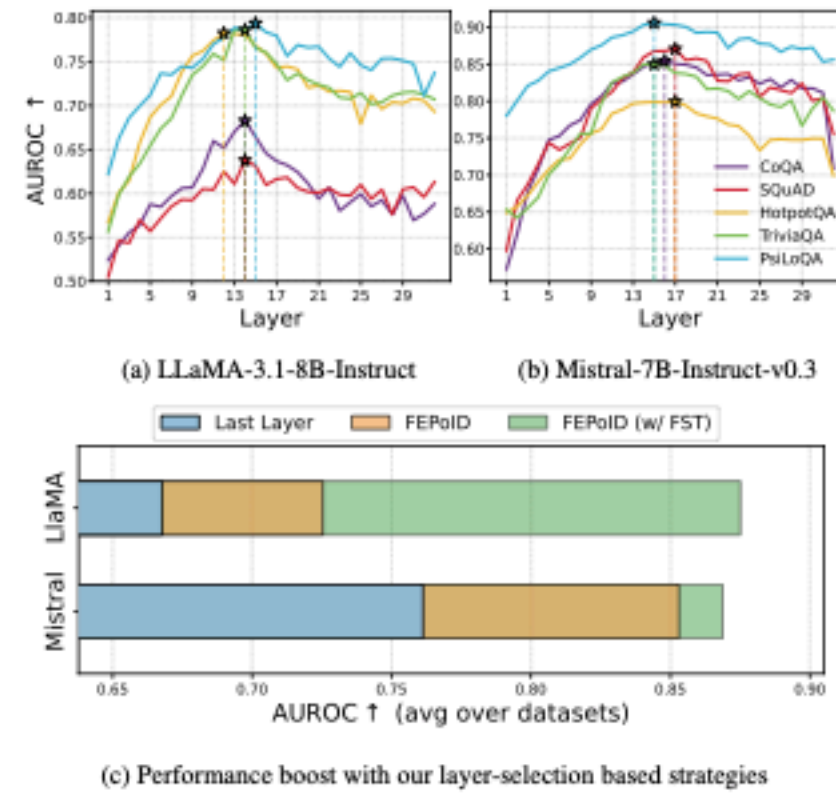


Figure 1. Hallucination detection performance under a unified experimental setting. For all experiments, we extract last-token representations from each layer and train an MLP classifier for hallucination detection. **(TOP):** Layer-wise AUROC under oracle training, where the best-performing layer (starred) consistently lies in the intermediate layers. **(Bottom):** Mean AUROC averaged across datasets under different layer-selection strategies. FEPoID consistently outperforms the last-layer heuristic, and its combination with the truncation strategy yields further gains across models.

ternally inconsistent outputs. Detecting such hallucinations without modifying or fine-tuning the underlying model is therefore an important practical problem (Huang et al., 2025; Farquhar et al., 2024; Li et al., 2024). Prior work has approached hallucination detection using uncertainty-based

ProbMoE: Differentiable Probabilistic Routing for Mixture-of-Experts

Heng Zhao¹ Zilei Shao² Guy Van den Broeck² Zhe Zeng¹

Abstract

Mixture-of-Experts (MoE) models scale by activating only a small subset of experts per token. However, training such models remains challenging because top- k routing is discrete and non-differentiable, requiring gradient estimators for expert selection whose design remains a central open problem. We introduce ProbMoE, a probabilistic routing framework that models expert selection as a distribution over cardinality-constrained expert subsets and formulates routing as probabilistic inference in this discrete subset space. We first propose ProbMoE Exact- k routing, which samples k -expert subsets in the forward pass, and the backward pass uses gradients through each expert’s exact marginal probability as a tractable surrogate for the true gradient. ProbMoE naturally generalizes to a dynamic- k routing setting, where both training and inference constrain the routing cardinality to the same predefined range, allowing adaptive expert allocation per token. Across benchmarks and model backbones, ProbMoE Exact- k achieves strong performance compared to competitive baselines, with improved expert utilization and routing diversity; ProbMoE Dynamic- k achieves comparable performance with fewer activated experts. Code is available at: <https://github.com/HengHugoZhao/ProbMoE.git>

of experts for each token, MoE models achieve sublinear growth in FLOPs and allow the total parameter count to substantially exceed the active compute budget (Du et al., 2022; Liu et al., 2023; Jiang et al., 2024).

However, training MoE models remains challenging because routing relies on a hard top- k operator that is discrete and non-differentiable, making it incompatible with standard gradient-based updates. In standard MoE architectures, the router computes expert scores by applying a softmax operation on expert logits and then selects the top- k experts per token. To bypass differentiating the top- k operator, common training procedures ignore its dependence on the router logits and propagate gradients only through the softmax probabilities (Shazeer et al., 2017; Lepikhin et al., 2021; Rajbhandari et al., 2022). As a result, the router receives limited learning signal about alternative expert subsets beyond the deterministically selected top- k experts, which can lead to highly concentrated routing distributions, poor expert utilization, and unstable training dynamics (Lewis et al., 2021; Zuo et al., 2022; Clark et al., 2022).

These challenges call for a principled routing mechanism. Instead of treating routing as a deterministic operator with heuristic gradient approximations, we propose to cast expert routing as a probabilistic inference problem: for each token, the selection scores of each expert induce a distribution over expert subsets, and selecting k experts corresponds to probabilistic inference under a cardinality constraint. From this perspective, training aims to optimize the expected loss under the expert subset distribution, which requires